Data Science — szybki kurs Pythona

Część III. Wbudowane struktury danych, funkcje i pliki

Cel zajęć

Celem ćwiczeń jest poznanie najważniejszych wbudowanych struktur danych Pythona oraz sposobów ich przetwarzania.

Szczególną uwagę poświęcimy konstrukcjom, które będą później często występowały podczas pracy z bibliotekami **NumPy** i **pandas**.

Po zajęciach powinieneś umieć:

- tworzyć i modyfikować listy,
- korzystać z indeksowania i wycinków,
- rozumieć różnicę między listą i krotką,
- korzystać ze słowników i zbiorów,
- sprawdzać obecność elementów za pomocą `in`,
- iterować po danych,
- korzystać z `enumerate()` i `zip()`,
- tworzyć listy za pomocą list comprehensions,
- definiować własne funkcje,
- korzystać z argumentów i wartości domyślnych,
- rozumieć prostą funkcję `lambda`,
- zapisywać dane do pliku i odczytywać je z pliku.

Każdy przykład wpisuj samodzielnie do kolejnej komórki Google Colab.

Komórkę uruchamiamy:

Shift + Enter

---

1. Lista — `list`

Lista pozwala przechowywać wiele wartości w jednym obiekcie.

Utwórz:

```
temperatures = [18.5, 21.0, 19.5, 24.0, 17.5]

print(temperatures)
print(type(temperatures))
```

Sprawdź liczbę elementów:

```
print(len(temperatures))
```

Elementy listy mogą mieć również różne typy:

```
data = ["Python", 10, 3.14, True]

print(data)
```

W analizie danych zazwyczaj będziemy jednak pracowali z kolekcjami zawierającymi wartości tego samego rodzaju.

Małe zadanie

Utwórz listę:

```
scores = [72, 55, 91, 48, 83]
```

Wyświetl:

- całą listę,
- jej typ,
- liczbę znajdujących się w niej elementów.

2. Indeksowanie listy

Elementy listy są numerowane od `0`.

Utwórz:

```
languages = ["Python", "C++", "Java", "R"]
```

Sprawdź:

```
print(languages[0])
print(languages[1])
print(languages[2])
```

Pytanie

Co zwróci:

```
languages[3]
```

?

Sprawdź również:

```
print(languages[-1])
print(languages[-2])
```

Ujemne indeksy pozwalają liczyć elementy od końca.

Małe zadanie

Dla:

```
values = [10, 20, 30, 40, 50]
```

wyświetl:

- pierwszy element,
- trzeci element,
- ostatni element,
- przedostatni element.

3. Lista jest modyfikowalna

Element listy możemy zmienić.

```
numbers = [10, 20, 30, 40]

numbers[1] = 200

print(numbers)
```

Możemy również dodawać elementy:

```
numbers.append(50)

print(numbers)
```

Sprawdź:

```
numbers.insert(1, 15)

print(numbers)
```

Usuń element:

```
numbers.remove(30)

print(numbers)
```

Sprawdź również:

```
last = numbers.pop()

print(last)
print(numbers)
```

Pytanie

Czym różnią się:

`append()`

oraz:

`insert()`

?

Małe zadanie

Utwórz:

```
cities = ["Warszawa", "Kraków", "Gdańsk"]
```

Następnie:

1. dodaj `"Poznań"` na końcu,
2. dodaj `"Łódź"` na pozycji `1`,
3. usuń `"Kraków"`,
4. wyświetl końcową listę.

---

4. Wycinki — slicing

Możemy pobierać fragmenty listy.

Utwórz:

```
data = [10, 20, 30, 40, 50, 60]
```

Sprawdź:

```
print(data[1:4])
```

Pytanie

Które elementy zostały wyświetlone?

W zapisie:

`data[1:4]`

element o indeksie `1` należy do wyniku, natomiast element o indeksie `4` już nie.

Sprawdź:

```
print(data[:3])
print(data[3:])
```

Teraz:

```
print(data[::2])
```

oraz:

```
print(data[::-1])
```

Obserwacja

Ogólny zapis wycinka ma postać:

```
dane[początek:koniec:krok]
```

Nie musimy podawać wszystkich trzech wartości.

Małe zadanie

Dla:

```
values = [1, 2, 3, 4, 5, 6, 7, 8]
```

wyświetl:

- pierwsze trzy elementy,
- elementy od czwartego do końca,
- co drugi element,
- całą listę w odwrotnej kolejności.

---

5. Przypisanie a kopiowanie listy

Przypomnijmy sobie przykład z poprzednich zajęć:

```
a = [1, 2, 3]
b = a

b.append(4)

print(a)
print(b)
```

Pytanie

Dlaczego zmieniły się obie zmienne?

Instrukcja:

```
b = a
```

nie tworzy nowej listy.

Obie nazwy wskazują na ten sam obiekt.

Sprawdź teraz:

```
a = [1, 2, 3]
b = a.copy()

b.append(4)

print(a)
print(b)
```

Obserwacja

Tym razem `a` pozostało niezmienione.

Małe zadanie

Utwórz listę:

```
original = [10, 20, 30]
```

Utwórz jej kopię o nazwie:

```
copy
```

Do kopii dodaj wartość `40`.

Sprawdź zawartość obu list.

---

6. Krotka — `tuple`

Krotka przypomina listę, ale jest **niemodyfikowalna**.

Utwórz:

```
point = (10, 20)
print(point)
print(type(point))
```

Elementy odczytujemy tak samo jak z listy:

```
print(point[0])
print(point[1])
```

Spróbuj:

```
point[0] = 100
```

Przeczytaj komunikat błędu.

Obserwacja

Lista:

```
[10, 20]
```

jest modyfikowalna.

Krotka:

```
(10, 20)
```

jest niemodyfikowalna.

---

7. Rozpakowywanie krotek

Python pozwala przypisać poszczególne elementy krotki do kilku zmiennych.

```
point = (10, 20)

x, y = point

print(x)
print(y)
```

Możemy zapisać jeszcze krócej:

```
x, y = 10, 20

print(x)
print(y)
```

Sprawdź ciekawy sposób zamiany wartości dwóch zmiennych:

```
a = 5
b = 10

a, b = b, a
print(a)
print(b)
```

Małe zadanie

Dla:

```
student = ("Anna", 82)
```

rozpakuj dane do zmiennych:

```
name
score
```

a następnie je wyświetl.

---

8. Słownik — `dict`

Słownik przechowuje dane w postaci:

klucz → wartość

Utwórz:

```
student = {
    "name": "Anna",
    "score": 82,
    "passed": True
}

print(student)
print(type(student))
```

Odczytaj pojedynczą wartość:

```
print(student["name"])
print(student["score"])
```

Pytanie

Czy w słowniku odwołujemy się do wartości za pomocą pozycji `0`, `1`, `2`?

Nie. Korzystamy z **kluczy**.

9. Modyfikowanie słownika

Zmień wynik:

```
student["score"] = 90

print(student)
```

Dodaj nową informację:

```
student["group"] = "Data Science"

print(student)
```

Sprawdź:

```
print(student.keys())
print(student.values())
print(student.items())
```

Małe zadanie

Utwórz słownik opisujący pomiar:

```
measurement = {
    "city": "Warszawa",
    "temperature": 18.5,
    "humidity": 65
}
```

Następnie:

1. wyświetl temperaturę,
2. zmień temperaturę na `20.0`,
3. dodaj klucz `"pressure"` z wartością `1013`,
4. wyświetl cały słownik.

---

10. Iterowanie po słowniku

Utwórz:

```
student = {
    "name": "Anna",
    "score": 82,
    "passed": True
}
```

Sprawdź:

```
for key in student:
    print(key)
```

Teraz:

```
for value in student.values():
    print(value)
```

Najbardziej przydatna będzie często konstrukcja:

```
for key, value in student.items():
    print(key, value)
```

Obserwacja

Metoda:

```
items()
```

udostępnia jednocześnie klucz i odpowiadającą mu wartość.

Małe zadanie

Dla:

```
weather = {
    "temperature": 21,
    "humidity": 60,
    "pressure": 1012
}
```

wyświetl każdą parę:

```
klucz wartość
```

za pomocą pętli `for`.

---

11. Zbiór — `set`

Zbiór przechowuje **unikalne wartości**.

Sprawdź:

```
numbers = {1, 2, 3, 3, 3, 2, 1}

print(numbers)
print(type(numbers))
```

Pytanie

Co stało się z powtarzającymi się wartościami?

Praktyczny przykład:

```
cities = ["Warszawa", "Kraków", "Warszawa", "Gdańsk", "Kraków"]

unique_cities = set(cities)

print(unique_cities)
```

Dodaj element:

```
unique_cities.add("Poznań")

print(unique_cities)
```

---

12. Operacje na zbiorach

Utwórz:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
```

Sprawdź część wspólną:

```
print(a & b)
```

Sprawdź sumę zbiorów:

```
print(a | b)
```

Sprawdź różnicę:

```
print(a - b)
```

Małe zadanie

Dane są dwa zbiory:

```
group_a = {"Anna", "Jan", "Ola"}
group_b = {"Ola", "Adam", "Jan"}
```

Znajdź:

- osoby występujące w obu grupach,
- wszystkie osoby występujące przynajmniej w jednej grupie.

13. Operator `in`

Operator `in` pozwala sprawdzić, czy element znajduje się w kolekcji.

```
numbers = [10, 20, 30, 40]

print(20 in numbers)
print(100 in numbers)
```

Działa również dla napisów:

```
print("Python" in ["Python", "C++", "R"])
```

oraz słowników:

```
student = {
    "name": "Anna",
    "score": 82
}

print("name" in student)
print("age" in student)
```

Obserwacja

W przypadku słownika operator `in` sprawdza obecność **klucza**.

Możemy również użyć:

```
not in
```

Sprawdź:

```
print("age" not in student)
```

Małe zadanie

Dla:

```
languages = ["Python", "R", "C++"]
```

sprawdź, czy na liście znajdują się:

- `"Python"`,
- `"Java"`.

14. `enumerate()` — indeks i wartość

Czasami potrzebujemy zarówno wartości, jak i jej pozycji.

Możemy napisać:

```
languages = ["Python", "C++", "R"]

for i in range(len(languages)):
    print(i, languages[i])
```

Python oferuje jednak wygodniejsze rozwiązanie:

```
for i, language in enumerate(languages):
    print(i, language)
```

Obserwacja

`enumerate()` zwraca jednocześnie:

- numer elementu,
- jego wartość.

Małe zadanie

Dla:

```
scores = [72, 85, 91, 66]
```

uzyskaj:

```
0 72
1 85
2 91
3 66
```

za pomocą `enumerate()`.

---

15. `zip()` — łączenie kilku sekwencji

Mamy dwie listy:

```
names = ["Anna", "Jan", "Ola"]
scores = [80, 65, 91]
```

Chcemy pracować jednocześnie z odpowiadającymi sobie elementami.

Sprawdź:

```
for name, score in zip(names, scores):
    print(name, score)
```

Obserwacja

`zip()` łączy elementy znajdujące się na odpowiadających sobie pozycjach.

Możemy zobaczyć wynik również jako listę:

```
pairs = list(zip(names, scores))

print(pairs)
```

Pytanie

Jaki typ mają poszczególne elementy listy `pairs`?

Małe zadanie

Dane są:

```
cities = ["Warszawa", "Kraków", "Gdańsk"]
temperatures = [20.5, 22.0, 18.5]
```

Za pomocą `zip()` wypisz:

```
Warszawa 20.5
Kraków 22.0
Gdańsk 18.5
```

16. List comprehension — tworzenie list w stylu Pythona

Załóżmy, że chcemy utworzyć listę kwadratów liczb.

Możemy napisać:

```
squares = []

for x in range(5):
    squares.append(x ** 2)
print(squares)
```

Python pozwala zapisać to krócej:

```
squares = [x ** 2 for x in range(5)]
print(squares)
```

Taki zapis nazywamy **list comprehension**.

Obserwacja

Porównaj:

```
for x in range(5):
    ...
```

z:

```
[x ** 2 for x in range(5)]
```

Małe zadanie

Za pomocą list comprehension utwórz listę:

```
[10, 20, 30, 40, 50]
```

na podstawie:

```
range(1, 6)
```

---

17. List comprehension z warunkiem

Utwórz:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
```

Klasycznie liczby parzyste możemy znaleźć tak:

```
even = []

for number in numbers:
    if number % 2 == 0:
        even.append(number)

print(even)
```

Teraz zapisz to w jednej instrukcji:

```
even = [number for number in numbers if number % 2 == 0]

print(even)
```

Małe zadanie

Dane są wyniki:

```
scores = [42, 75, 51, 33, 90, 68]
```

Za pomocą list comprehension utwórz listę zawierającą tylko wyniki wynoszące co najmniej `50`.

18. Funkcje — `def`

Funkcja pozwala nadać nazwę fragmentowi kodu, który możemy później wielokrotnie wykorzystywać.

Utwórz:

```
def square(x):
    return x ** 2
```

Wywołaj:

```
print(square(5))
print(square(10))
```

Obserwacja

W definicji:

```
def square(x):
```

`x` jest argumentem funkcji.

Instrukcja:

```
return
```

zwraca wynik.

Małe zadanie

Napisz funkcję:

```
double(x)
```

która zwraca dwukrotność przekazanej liczby.

Sprawdź ją dla:

```
double(5)
double(12)
```

19. Funkcja z kilkoma argumentami

Utwórz:

```
def add(a, b):
    return a + b
```

Sprawdź:

```
print(add(3, 5))
print(add(10, 20))
```

Możemy również używać argumentów nazwanych:

```
print(add(a=10, b=5))
```

Małe zadanie

Napisz funkcję:

```
average(a, b)
```

zwracającą średnią dwóch liczb.

---

20. Argumenty z wartościami domyślnymi

Funkcja może posiadać argument z wartością domyślną.

Utwórz:

```
def calculate_price(price, tax=0.23):
    return price * (1 + tax)
```

Sprawdź:

```
print(calculate_price(100))
```

Teraz:

```
print(calculate_price(100, 0.08))
```

Pytanie

Jaką wartość parametru `tax` wykorzystano w pierwszym wywołaniu?

A jaką w drugim?

Małe zadanie

Napisz funkcję:

```
power(x, exponent=2)
```

która domyślnie zwraca kwadrat liczby.

Sprawdź:

```
power(5)
power(5, 3)
```

---

21. Funkcja może zwrócić kilka wartości

Utwórz:

```
def min_max(values):
    return min(values), max(values)
```

Sprawdź:

```
numbers = [7, 2, 19, 4, 12]

result = min_max(numbers)

print(result)
print(type(result))
```

Pytanie

Jaki typ został zwrócony?

Możemy od razu rozpakować wynik:

```
minimum, maximum = min_max(numbers)

print(minimum)
print(maximum)
```

Małe zadanie

Napisz funkcję:

```
first_last(values)
```

która zwraca pierwszy i ostatni element listy.

22. Funkcja jako obiekt

W Pythonie funkcje również są obiektami.

Utwórz:

```
def square(x):
    return x ** 2
```

Następnie:

```
f = square

print(f(5))
```

Obserwacja

Nie zapisaliśmy:

```
f = square()
```

lecz:

```
f = square
```

Do zmiennej `f` przypisaliśmy funkcję.

To będzie później przydatne podczas pracy z metodami przetwarzającymi dane.

23. Funkcja skrócona `lambda`

Prosta funkcja:

```
def square(x):
    return x ** 2
```

może zostać zapisana jako:

```
square = lambda x: x ** 2
```

Sprawdź:

```
print(square(5))
```

Jeszcze jeden przykład:

```
add = lambda a, b: a + b

print(add(3, 4))
```

Obserwacja

`lambda` służy do tworzenia krótkich funkcji anonimowych.

W analizie danych możemy później spotkać konstrukcje podobne do:

```
lambda x: x * 2
```

czy:

```
lambda x: x.upper()
```

Małe zadanie

Utwórz funkcję `lambda`, która przyjmuje liczbę i zwraca jej sześcian.

---

24. Podstawowe funkcje przydatne podczas pracy z danymi

Utwórz:

```
values = [10, 5, 17, 8, 20]
```

Sprawdź:

```
print(len(values))
print(min(values))
print(max(values))
print(sum(values))
```

Średnią możemy policzyć:

```
average = sum(values) / len(values)

print(average)
```

Posortuj dane:

```
print(sorted(values))
```

Sprawdź:

```
print(sorted(values, reverse=True))
```

Małe zadanie

Dla:

```
temperatures = [18.5, 22.0, 19.0, 25.5, 17.5]
```

oblicz:

- liczbę pomiarów,
- temperaturę minimalną,
- temperaturę maksymalną,
- średnią temperaturę.

---

25. Zapis do pliku

Utwórz plik tekstowy:

```
with open("data.txt", "w") as file:
    file.write("Python\n")
    file.write("Data Science\n")
    file.write("NumPy\n")
```

Po wykonaniu komórki w panelu plików Google Colab powinien pojawić się plik:

```
data.txt
```

Obserwacja

Konstrukcja:

```
with open(...)
```

otwiera plik i automatycznie zamyka go po zakończeniu bloku.

Tryb:

```
"w"
```

oznacza zapis.

26. Odczyt pliku

Odczytaj utworzony plik:

```
with open("data.txt", "r") as file:
    text = file.read()

print(text)
```

Tryb:

```
"r"
```

oznacza odczyt.

Możemy również czytać plik linia po linii:

```
with open("data.txt", "r") as file:
    for line in file:
        print(line)
```

Pytanie

Dlaczego między liniami mogą pojawić się dodatkowe odstępy?

Spróbuj:

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Małe zadanie

Utwórz plik:

```
languages.txt
```

zawierający trzy języki programowania.

Następnie odczytaj go i wyświetl każdą linię.

27. Ćwiczenie podsumowujące

Mamy dane:

```
students = [
    ("Anna", 78),
    ("Jan", 43),
    ("Ola", 91),
    ("Piotr", 68),
    ("Adam", 39)
]
```

Najpierw wyświetl wszystkie dane:

```
for student in students:
    print(student)
```

Teraz wykorzystaj rozpakowywanie:

```
for name, score in students:
    print(name, score)
```

Zadanie 1

Wypisz tylko osoby, które uzyskały co najmniej `50` punktów.

Zadanie 2

Utwórz listę samych wyników.

Możesz wykorzystać:

```
scores = [score for name, score in students]
```

Wyświetl ją.

Zadanie 3

Na podstawie listy `scores` oblicz:

- najmniejszy wynik,
- największy wynik,
- średni wynik.

Zadanie 4

Za pomocą list comprehension utwórz listę nazwisk osób, które zaliczyły:

```
passed = [
    name
    for name, score in students
    if score >= 50
]
```

Wyświetl:

```
print(passed)
```

Pytanie

Ile osób zaliczyło?
Spróbuj odpowiedzieć za pomocą:

```
len(passed)
```

28. Ćwiczenie końcowe — przeczytaj kod

Najpierw nie uruchamiaj programu.

Przeczytaj:

```
data = [12, 5, 18, 7, 21, 4]

selected = [x for x in data if x >= 10]

minimum = min(selected)
maximum = max(selected)
average = sum(selected) / len(selected)

print(selected)
print(minimum)
print(maximum)
print(average)
```

Odpowiedz:

1. Jaką listę utworzy `selected`?
2. Jakie wartości zostaną pominięte?
3. Jaką wartość otrzyma `minimum`?
4. Jaką wartość otrzyma `maximum`?
5. Jaką wartość otrzyma `average`?

Dopiero teraz uruchom kod i sprawdź odpowiedzi.

29. Ćwiczenie końcowe — mini analiza danych

Mamy temperatury zmierzone w kolejnych dniach:

```
temperatures = [
    18.5,
    21.0,
    19.5,
    24.0,
    17.5,
    22.5,
    25.0
]
```

Wykonaj kolejno następujące zadania.

Zadanie 1

Wyświetl liczbę pomiarów.

Zadanie 2

Wyświetl:

- temperaturę minimalną,
- temperaturę maksymalną,
- średnią temperaturę.

Zadanie 3

Za pomocą pętli `for` wypisz temperatury większe od `20`.

Zadanie 4

Za pomocą list comprehension utwórz:

`warm_days`

zawierającą tylko temperatury większe od `20`.

Zadanie 5

Policz liczbę ciepłych dni:

```
len(warm_days)
```

Zadanie 6

Napisz funkcję:

```
def average(values):
```

która zwraca średnią wartości z listy.

Użyj jej dla listy:

```
temperatures
```

Zadanie 7

Zapisz wszystkie temperatury do pliku:

```
temperatures.txt
```

Każda temperatura powinna znaleźć się w osobnej linii.

Zadanie 8

Odczytaj utworzony plik i wyświetl jego zawartość.

---

30. Podsumowanie

Po wykonaniu ćwiczeń powinieneś rozumieć kod:

```
students = [
    ("Anna", 78),
    ("Jan", 43),
    ("Ola", 91),
    ("Piotr", 68)
]

passed = [
    name
    for name, score in students
    if score >= 50
]

print(passed)
```

oraz:

```
def average(values):
    return sum(values) / len(values)

scores = [78, 43, 91, 68]

print(average(scores))
```

Odpowiedz na pytania:

1. Czym różni się lista od krotki?
2. Jak odczytać pierwszy i ostatni element listy?
3. Co oznacza zapis `data[1:4]`?
4. Jak utworzyć kopię listy?
5. Do czego służy słownik?
6. Czym różni się klucz od wartości w słowniku?
7. Do czego służy zbiór `set`?
8. Co sprawdza operator `in`?
9. Do czego służy `enumerate()`?
10. Do czego służy `zip()`?
11. Co oznacza list comprehension?
12. Do czego służy `def`?
13. Jak zwrócić wynik z funkcji?
14. Co oznacza argument z wartością domyślną?
15. Jak funkcja może zwrócić kilka wartości?
16. Czym jest prosta funkcja `lambda`?
17. Do czego służą `len()`, `min()`, `max()` i `sum()`?
18. Co oznacza konstrukcja `with open(...)`?
19. Czym różnią się tryby `"r"` i `"w"`?
20. Dlaczego poznane konstrukcje będą przydatne podczas pracy z NumPy i pandas?